# Building Footprint Segmentation from Aerial Imagery

PhD Technical Assessment, Part 2, Research Summary Report

## Dataset and Partitioning

The Inria Aerial Image Labeling Dataset provides 180 labeled training tiles spread across five cities: Austin, Chicago, Kitsap County, Western Tyrol, and Vienna, with 36 tiles each. A separate set of 180 test tiles also exists, but its ground truth masks were never released publicly, since the dataset began as a competition and the organizers kept those answers private. For that reason, all training and evaluation in this work uses only the 180 labeled training tiles, divided into custom training and validation sets.

Rather than shuffling all 180 tiles together at random, one entire city, Chicago, was held out for validation, and the model was trained on the remaining four cities, 144 tiles in total. This was chosen deliberately over a random split, because random shuffling risks placing two nearly identical neighboring patches from the same city on opposite sides of the split. The model would then see something very close to its own validation data during training, and the results would look better than they really are. Holding out a whole city removes that risk entirely and tests the question that actually matters: can the model recognize buildings in a place it has never seen at all. This also mirrors how the dataset's own original train and test sets were built, which kept certain cities out of training entirely.

Each tile is 5000 by 5000 pixels, far too large to feed into a segmentation network directly, so every tile was cut into 512 by 512 patches. Patch extraction was carried out after the city split, not before, so a patch cut from a training city can never physically overlap with a patch from the held out validation city. The way files were read from disk also needed careful thought. The first version of the data loader reread a full 5000 by 5000 tile every time it needed a single small patch, which made one training epoch painfully slow. This was fixed by reading each tile from disk exactly once and pulling several random patches out of it before moving to the next tile, which brought one epoch down from what looked like hours to about ten to twelve minutes.

## Model Architecture and Loss Function

The model is a U-Net with a ResNet34 encoder pretrained on ImageNet, giving 24.4 million parameters in total. U-Net was selected because of how it is built. The encoder half of the network repeatedly shrinks the image, trading away fine spatial detail in exchange for a wider view that lets the network recognize what it is looking at, for example that a patch of texture is a rooftop. The decoder half then expands that understanding back to full resolution to say exactly where each building is. The weakness of a plain shrink then expand design is that fine detail lost during shrinking cannot really be recovered later, much like a photocopy of a photocopy keeps losing quality. U-Net's skip connections address this by carrying a copy of the detailed features from each shrinking step directly across to the matching expanding step, so precise edge information does not have to survive the whole round trip through the bottleneck. Using a ResNet34 encoder that had already been trained on roughly a million ordinary photographs, rather than starting from random weights, gave the network a head start on generic visual features like edges and textures before it ever saw an aerial image, which mattered given the limited training budget.

Buildings usually cover a minority of the pixels in any given tile, so the loss function needed to prevent the model from simply predicting background everywhere. A combination of Dice loss and binary cross entropy was used. Dice loss measures how much the predicted building area overlaps with the true building area, and because that calculation is normalized by the size of the building regions themselves, it is not diluted by the much larger background area the way a plain per pixel loss would be. Dice loss on its own can be numerically unstable very

early in training, when the model's first predictions are close to all zero, so it was combined in equal parts with binary cross entropy, which gives smoother and more stable gradients from the very first batch. This combination is a well established approach in the segmentation literature.

A second version of the loss was also tried to target edge precision more directly, since Dice loss only helps with boundaries indirectly. A weight map was built for each patch using two standard image operations, dilation and erosion, and the difference between them isolates a thin ring of pixels sitting on each building's true boundary. Those boundary pixels were multiplied by five inside the cross entropy term, so the loss punishes mistakes at edges more heavily than mistakes in the interior of a building or in open background. This boundary weighted version was trained for the same ten epochs and compared against the original. The two loss functions land on different numeric scales, so their raw loss values are not comparable, but the resulting IoU and F1 scores are. The boundary weighted version came out slightly behind, reaching an IoU of 0.610 at its best epoch against 0.642 for the original. It is worth being honest about what can and cannot be concluded from that. No random seed was fixed for either run, so some of the gap could simply be ordinary run to run randomness in how patches were sampled and how the final layer was initialized, rather than a real effect of the boundary weighting. Given that, the plain Dice plus binary cross entropy loss is reported as the primary result throughout the rest of this report, and the boundary weighted version is treated as a fair experiment that was evaluated honestly, not presented as an improvement it did not clearly demonstrate.

# Training Setup and Hardware

Training used the Adam optimizer at a learning rate of 1e-4 for 10 epochs, drawing 16 random patches from every tile in each epoch, with a batch size of 8. Training ran on an Intel Arc A580 GPU through PyTorch's native XPU backend, at roughly 10 to 12 minutes per epoch. Evaluation used Intersection over Union, usually written as IoU, and the F1 or Dice score, deliberately instead of relying on plain pixel accuracy. With buildings covering a minority of pixels, a model that predicted background everywhere and understood nothing about buildings would still score around 85 percent pixel accuracy, while scoring exactly zero on IoU and F1. That gap is the whole reason those two metrics matter more here than accuracy does.

# Results

The best epoch, the tenth and final one, reached an IoU of 0.642 and an F1 score of 0.780 on the held out Chicago validation set, with the lowest validation loss of the whole run at 0.279. Figure 1 plots the full training history. The left panel shows training loss falling smoothly and consistently across all ten epochs, which indicates the model was learning steadily throughout. The right panel, showing validation IoU and F1 on Chicago, is a lot noisier, including a sharp dip around epoch 6 where IoU fell back to 0.488 before recovering. This points to the learning rate being slightly too high for fully smooth convergence at that stage of training. It is also worth noting that performance was still climbing at epoch 10 rather than having clearly leveled off, so training for longer would probably have pushed these numbers higher still. The fixed ten epoch budget is therefore treated as a genuine limitation of this run, rather than epoch 10 being claimed as a true peak.

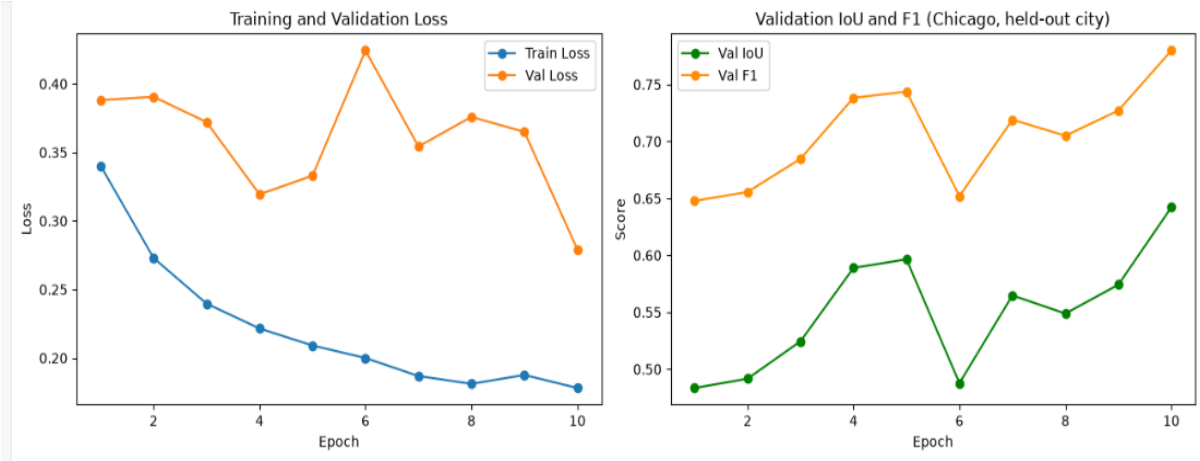



Figure 1: training and validation loss on the left, validation IoU and F1 on the held out city on the right, across 10 epochs.

# Qualitative Analysis: Success and Failure Cases

Beyond the aggregate numbers, specific predictions were inspected to understand where the model does well and where it struggles. The model was run on fixed, centered 512 by 512 patches from six Chicago tiles it had never trained on, and the per patch IoU ranged from 0.416 up to 0.934, a spread wide enough to be worth examining.

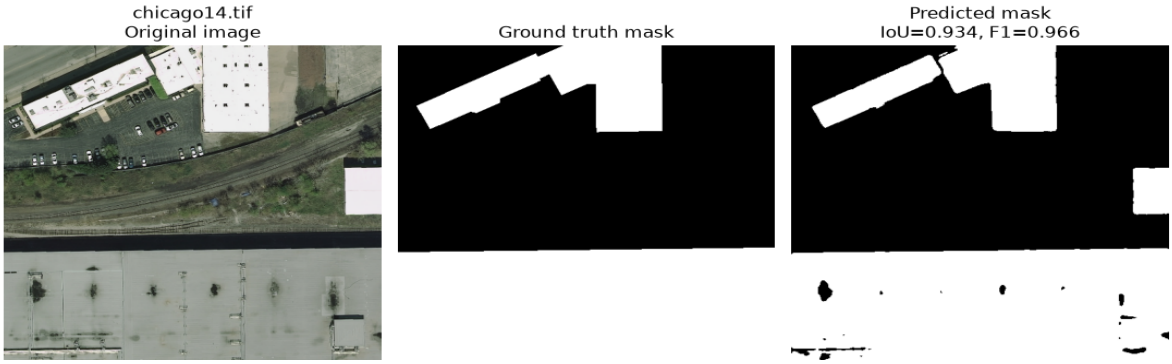



Figure 2: a success case, tile chicago14, IoU 0.934, F1 0.966, a large, simple, high contrast industrial building.

Figure 2 shows the strongest result. This is a single large industrial building with a clean, high contrast edge against the surrounding pavement, and the predicted mask matches the ground truth almost exactly, right down to correctly picking up a small separate structure elsewhere in the same patch. Large, simple, high contrast buildings like this one are close to a best case scenario for this kind of network.

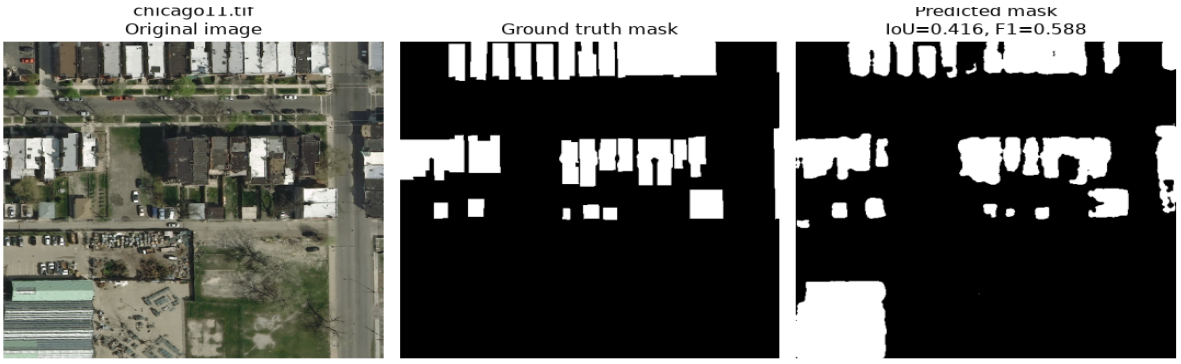



Figure 3: a failure case, tile chicago11, IoU 0.416, F1 0.588, a dense residential block of small, closely packed row houses.

Figure 3 shows the weakest result, and it is arguably the more informative of the two figures, since the way it fails is specific and explainable rather than a generic miss. Looking closely, the model does not hallucinate buildings that are not there, and it does not miss whole buildings either. It correctly works out roughly where the buildings are. What it gets wrong is that it merges several small, closely spaced row houses into one connected white blob, losing the narrow black gaps between individual houses that the ground truth mask clearly shows. This most likely comes down to the ResNet34 encoder's repeated downsampling discarding exactly the kind of fine spatial detail needed to keep two buildings that are only a few pixels apart visually separate. Skip connections help recover detail for large, well separated buildings like the one in Figure 2, but they are much less able to rescue detail that is only a few pixels wide to begin with.

# Summary and Discussion

In plain terms, this part of the assignment asked for a system that looks at an aerial photograph and draws an outline around every building in it, purely from the image itself, with no other information to go on. It is worth explaining why this is a hard problem rather than a simple one. Buildings come in every size, shape, and roof material imaginable, and they sit right next to things that can look similar from above, like parking lots, roads, and shadows. A model has to learn to tell all of that apart using nothing but color and shape patterns in a photograph.

The approach was to first make sure the model was being tested fairly. An entire city was set aside that the model would never see during training, so that when its performance was later measured, the result would reflect genuine skill at recognizing buildings rather than how well it memorized a particular neighborhood. A network design, U-Net, was then used that is specifically built to keep fine outline detail intact while it works out what it is looking at, together with a training setup that pushes the model to pay real attention to buildings, rather than taking the easy route of mostly predicting empty background, which would look deceptively accurate on paper without being useful.

After training, the model reached a respectable level of accuracy on a city it had never seen before, correctly outlining roughly two thirds of the true building area on average, by the measure segmentation researchers actually use for this kind of task. Looking at individual examples, it does very well on large, simple buildings with clear edges, and its main weak spot is dense neighborhoods of small houses packed close together, where it tends to blur several buildings into one shape instead of keeping them separate. A more advanced version of the training setup, one that specifically rewarded getting edges exactly right, was also tried, and it did not clearly beat the simpler version within the time available. That is itself a useful finding: a more complicated method is not automatically a better one, and it is worth actually testing that rather than assuming it.

If this work were continued, the clearest next step would be training for longer, since the results were still improving right up to the last epoch, and looking specifically at ways to help the model separate buildings that sit very close together, since that was the one consistent weak spot found.